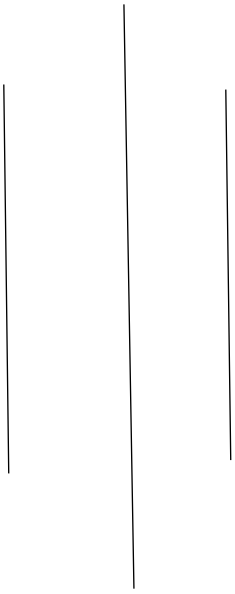
LA GRANDEE INTERNATIONAL COLLEGE

Simalchaur, Pokhara, Nepal



Lab Assignment

Of

"DSA (Data Structures & Algorithm)"

Submitted By

Divya Sharma

PU Registration no: 2025-1-53-0323

Semester: 3rd

Program: BCA

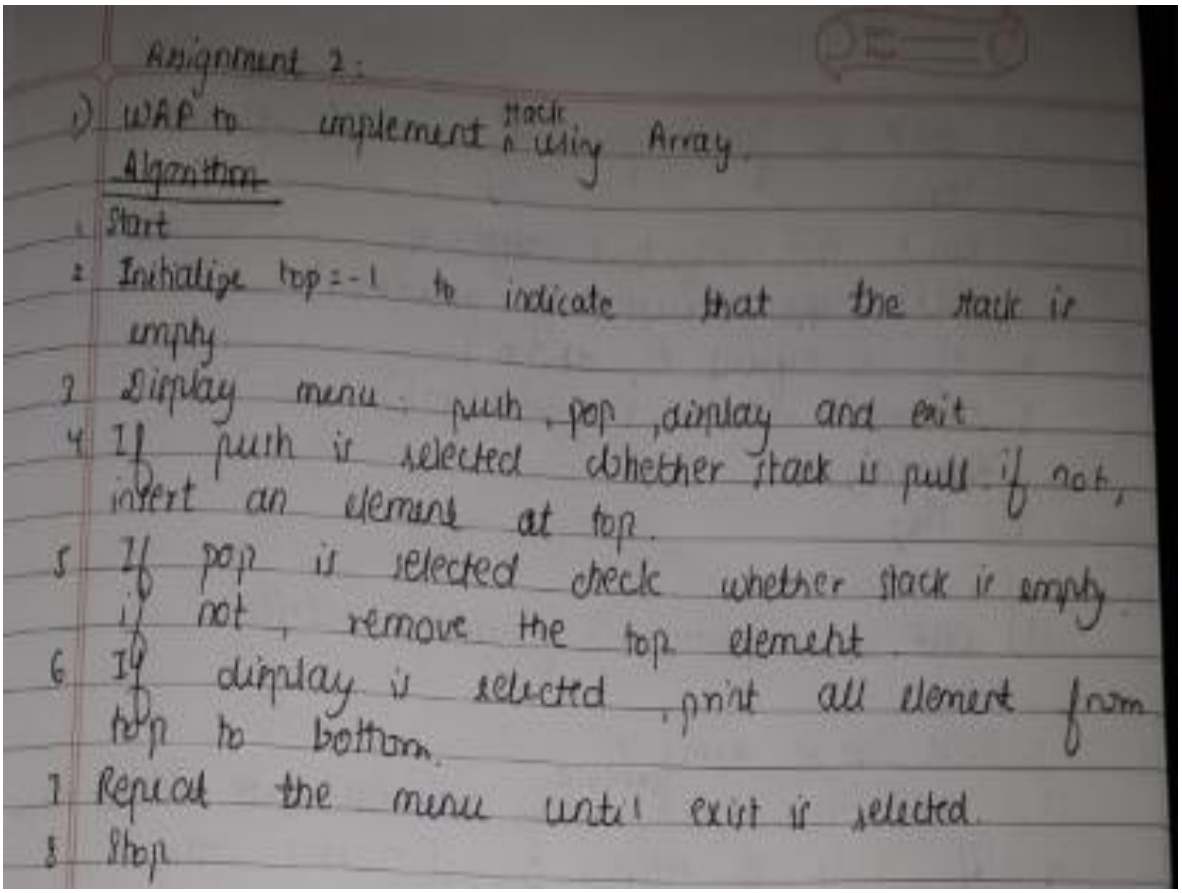
Submitted To

Ashwin Poudel

Divya Sharma (Registration no) : 25530417

Q1. Program to implement stack using array

Algorithm:



Assignment 2:

1) WAP to implement stack using Array.

Algorithm

1. Start
2. Initialize top = -1 to indicate that the stack is empty
3. Display menu: push, pop, display and exit
4. If push is selected whether stack is full if not, insert an element at top.
5. If pop is selected check whether stack is empty if not, remove the top element
6. If display is selected, print all element from top to bottom.
7. Repeat the menu until exit is selected.
8. Stop

Code:

```c
#include <stdio.h>

int main()
{
    int stack[5], top = -1;
    int choice, value, i;

    printf("Name: Divya Sharma\n");
    printf("Registration No: 2025-1-53-0323\n");

    do
    {
        printf("\n1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if(choice == 1)
        {
            if(top == 4)
            {
                printf("Stack Overflow\n");
            }
            else
            {
                printf("Enter value: ");
                scanf("%d", &value);

                top++;
                stack[top] = value;

                printf("Element pushed successfully\n");
            }
        }
        else if(choice == 2)
        {
            if(top == -1)
            {
                printf("Stack Underflow\n");
            }
            else
            {
                printf("Popped element = %d\n",
                    stack[top]);

                top--;
            }
```

```
        }
    }
    else if(choice == 3)
    {
        if(top == -1)
        {
            printf("Stack is empty\n");
        }
        else
        {
            printf("Stack elements:\n");

            for(i = top; i >= 0; i--)
            {
                printf("%d\n", stack[i]);
            }
        }
    }
    else if(choice == 4)
    {
        printf("Program ended\n");
    }
    else
    {
        printf("Invalid choice\n");
    }

} while(choice != 4);

return 0;
}
```

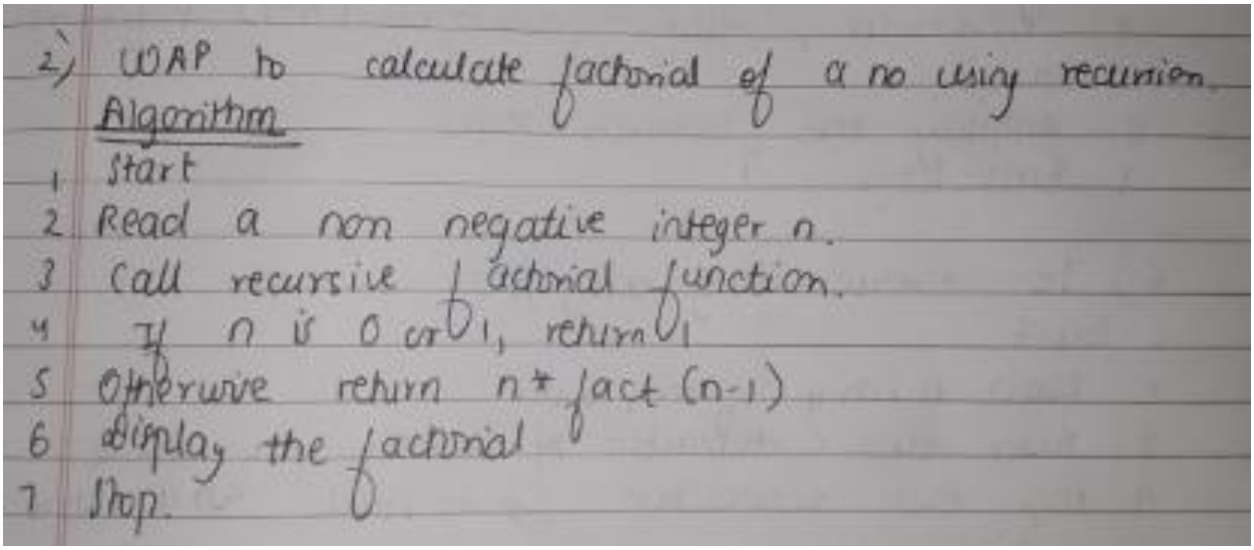
Output:

Result: The stack was successfully implemented using an array. The push, pop, and display operations were performed successfully.

- Time Complexity: Push and Pop – O(1); Display – O(n).
- Space Complexity: O(n), where (n) is the maximum size of the stack.

Q2. Program to calculate the factorial of a number using recursion

Algorithm:



Code:

```c
#include <stdio.h>

int fact(int n)
{
    if(n == 0)
    {
        return 1;
    }

    return n * fact(n - 1);
}

int main()
{
    int n, ans;

    printf("Name: Divya Sharma\n");
    printf("Registration No: 2025-1-53-0323\n");

    printf("Enter a number: ");
    scanf("%d", &n);

    if(n < 0 || n > 12)
    {
        printf("Enter a number between 0 and 12\n");
    }
    else
    {
        ans = fact(n);
        printf("Factorial = %d\n", ans);
    }

    return 0;
}
```

Output:

```
Name: Divya Sharma
Registration No: 2025-1-53-0323
Enter a number: 4
Factorial = 24

--------------------------------
```

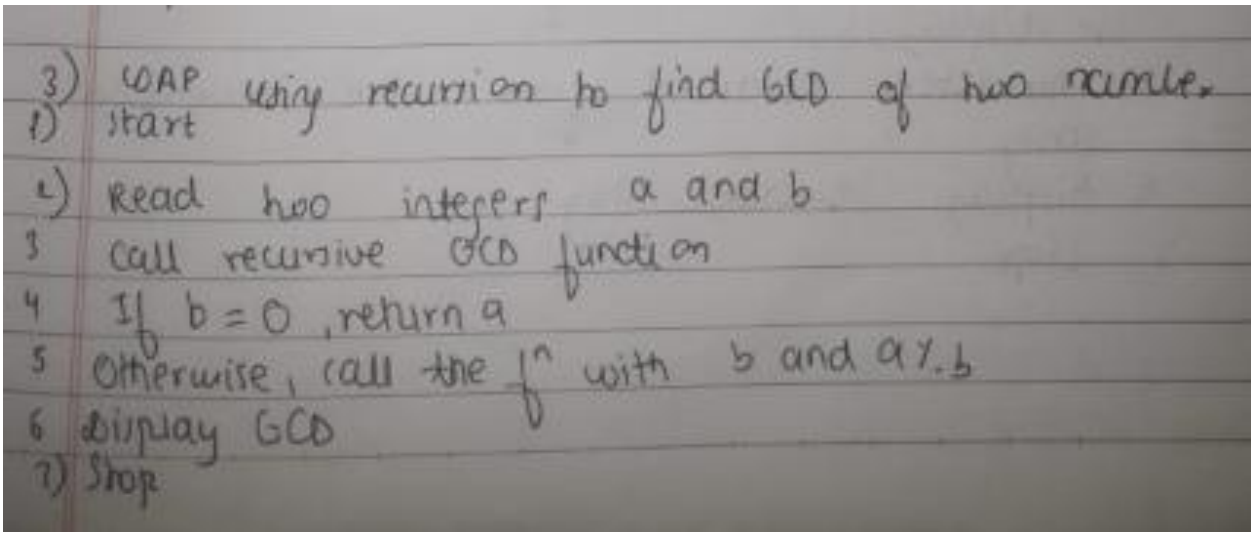
Result: The program successfully calculated the factorial of a given number using recursion.

Time complexity: O(n)

Space complexity: O(n), due to recursive function calls.

Q3. Program using recursion to find the GCD of two numbers.

Algorithm:



Code:

```c
#include <stdio.h>

int gcd(int a, int b)
{
    if(b == 0)
    {
        return a;
    }

    return gcd(b, a % b);
}

int main()
{
    int a, b, ans;

    printf("Name: Divya Sharma\n");
    printf("Registration No: 2025-1-53-0323\n");

    printf("Enter two positive numbers: ");
    scanf("%d %d", &a, &b);

    if(a <= 0 || b <= 0)
    {
        printf("Enter positive numbers only\n");
    }
    else
    {
        ans = gcd(a, b);
        printf("GCD = %d\n", ans);
    }

    return 0;
}
```

Output:

```
Name: Divya Sharma
Registration No: 2025-1-53-0323
Enter two positive numbers: 3
4
GCD = 1

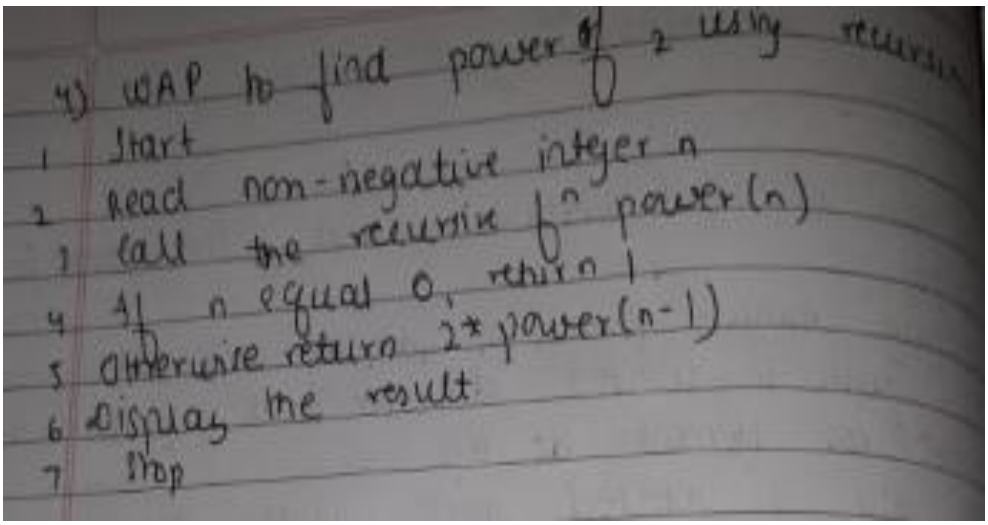
--------------------------------
Process exited after 10.05 seconds with return value 0
Press any key to continue . . .
```

Result: The program successfully calculated the Greatest Common Divisor (GCD) of two numbers using the recursive Euclidean algorithm.

- Time complexity: O(log(min(a, b))) for positive inputs.
- Space complexity: O(log(min(a, b))) for positive inputs, due to recursive calls.

Q4. To find the power of 2 using recursion.

Algorithm:



```c
#include <stdio.h>

int power(int n)
{
    if(n == 0)
    {
        return 1;
    }

    int a = power(n - 1);
    int b = 2 * a;

    return b;
}

int main()
{
    int n, ans;

    printf("Name: Divya Sharma\n");
    printf("Registration No: 2025-1-53-0323\n");

    printf("Enter the power: ");
    scanf("%d", &n);

    if(n < 0 || n > 30)
    {
        printf("Enter a number between 0 and 30\n");
    }
    else
    {
        ans = power(n);
        printf("Answer = %d\n", ans);
    }

    return 0;
}
```

Code:

Output:

```
Name: Divya Sharma
Registration No: 2025-1-53-0323
Enter the power: 3
Answer = 8

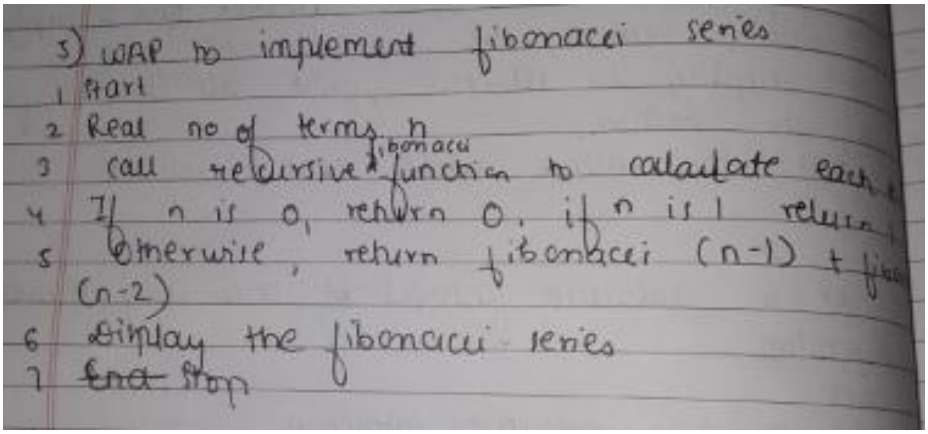
--------------------------------
```

Result: The program successfully calculated the power of 2 using recursion.

- **Time Complexity:** O(n).
- **Space Complexity:** O(n), due to recursive function calls.

Q5. To implement Fibonacci series using recursion.

Algorithm:



Code:

```c
#include <stdio.h>

int fib(int n)
{
    if(n == 0)
    {
        return 0;
    }

    if(n == 1)
    {
        return 1;
    }

    return fib(n - 1) + fib(n - 2);
}

int main()
{
    int n, i;

    printf("Name: Divya Sharma\n");
    printf("Registration No: 2025-1-53-0323\n");

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    if(n < 1 || n > 40)
    {
        printf("Enter a number between 1 and 40\n");
    }
    else
    {
        printf("Fibonacci series:\n");

        for(i = 0; i < n; i++)
        {
            printf("%d ", fib(i));
        }

        printf("\n");
    }

    return 0;
}
```

Output:

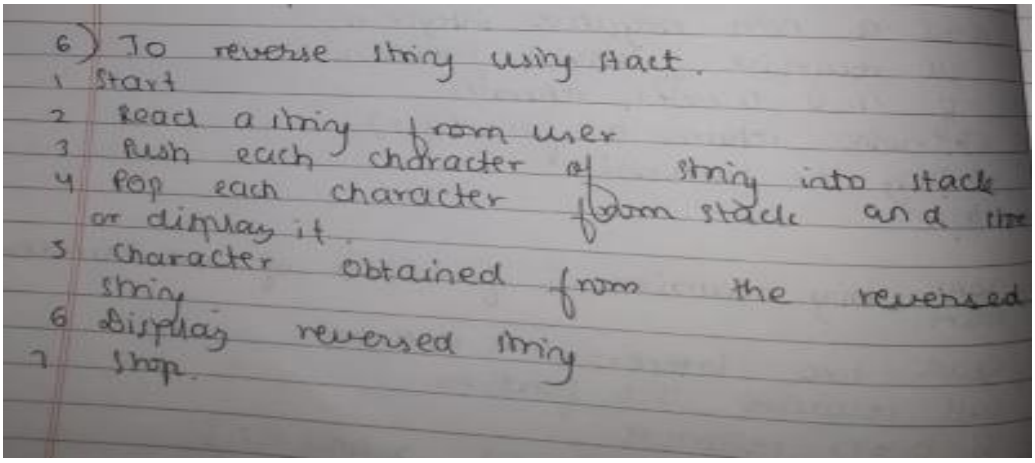
```
Name: Divya Sharma
Registration No: 2025-1-53-0323
Enter the number of terms: 4
Fibonacci series:
0 1 1 2
```

Result: The program successfully generated the Fibonacci series using recursion.

- **Time Complexity:** O(2^n) for the standard recursive Fibonacci function.
- **Space Complexity:** O(n), due to the maximum depth of recursive calls.

Q6. To reverse a string using stack.

Algorithm:



Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100], stack[100];
    int top = -1, i;

    printf("Name: Divya Sharma\n");
    printf("Registration No: 2025-1-53-0323\n");

    printf("Enter a string: ");
    scanf("%99s", str);

    for(i = 0; str[i] != '\0'; i++)
    {
        top++;
        stack[top] = str[i];
    }

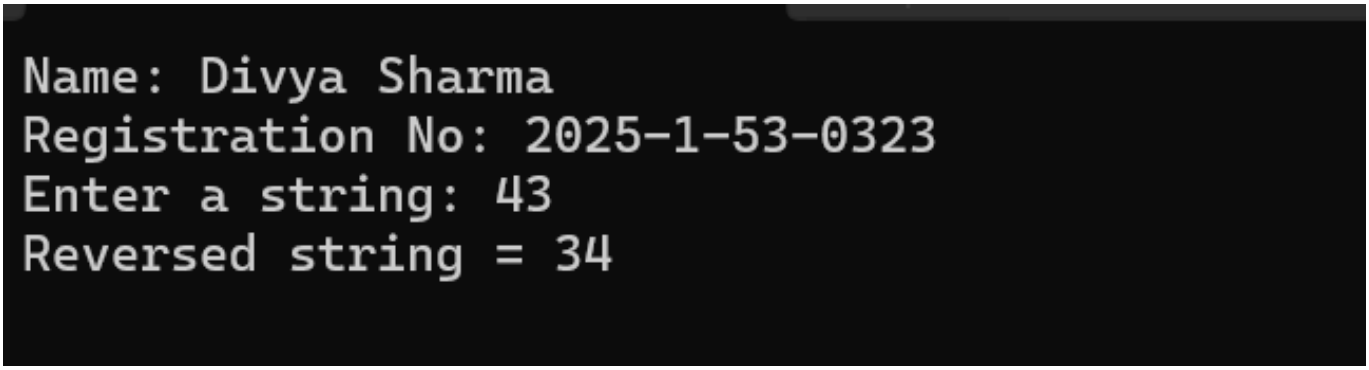
    for(i = 0; top >= 0; i++)
    {
        str[i] = stack[top];
        top--;
    }

    str[i] = '\0';

    printf("Reversed string = %s\n", str);

    return 0;
}
```

Output:



Result:

- **Time Complexity:** O(n), where n is the length of the string.
- **Space Complexity:** O(n), for storing the string and its characters in the stack.

7) Short notes on:

i) Tower of Hanoi

→ It is a mathematical tower that consist of three rods and no of disks of diff^n sizes.

Rules:

i) Only one disk can be moved at a time.

ii) Only the top disk can be moved.

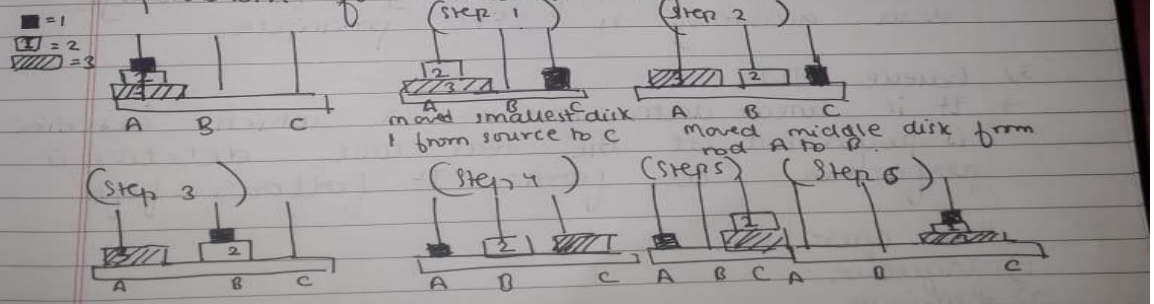
iii) A larger disk cannot be placed on smaller disk.

iv) All disk must be moved from source rod to destination rod using auxillary rod.

Working: Solved using recursion. First small are moved then large disk is moved to destination rod and finally smaller are moved onto it.

Min no of moved req for n disk is $T(n) = 2^n - 1$.

Conclusion: Tower of Hanoi is an important problem for understanding recursion



All three disk are transferred from rod A to C using B int the min^m of $2^3 - 1 = 7$ move.

2) Stack ADT (Abstract Data Type)
→ It is a linear data structure in which insertion, deletion are performed at same end called top. It follows LIFO principle.

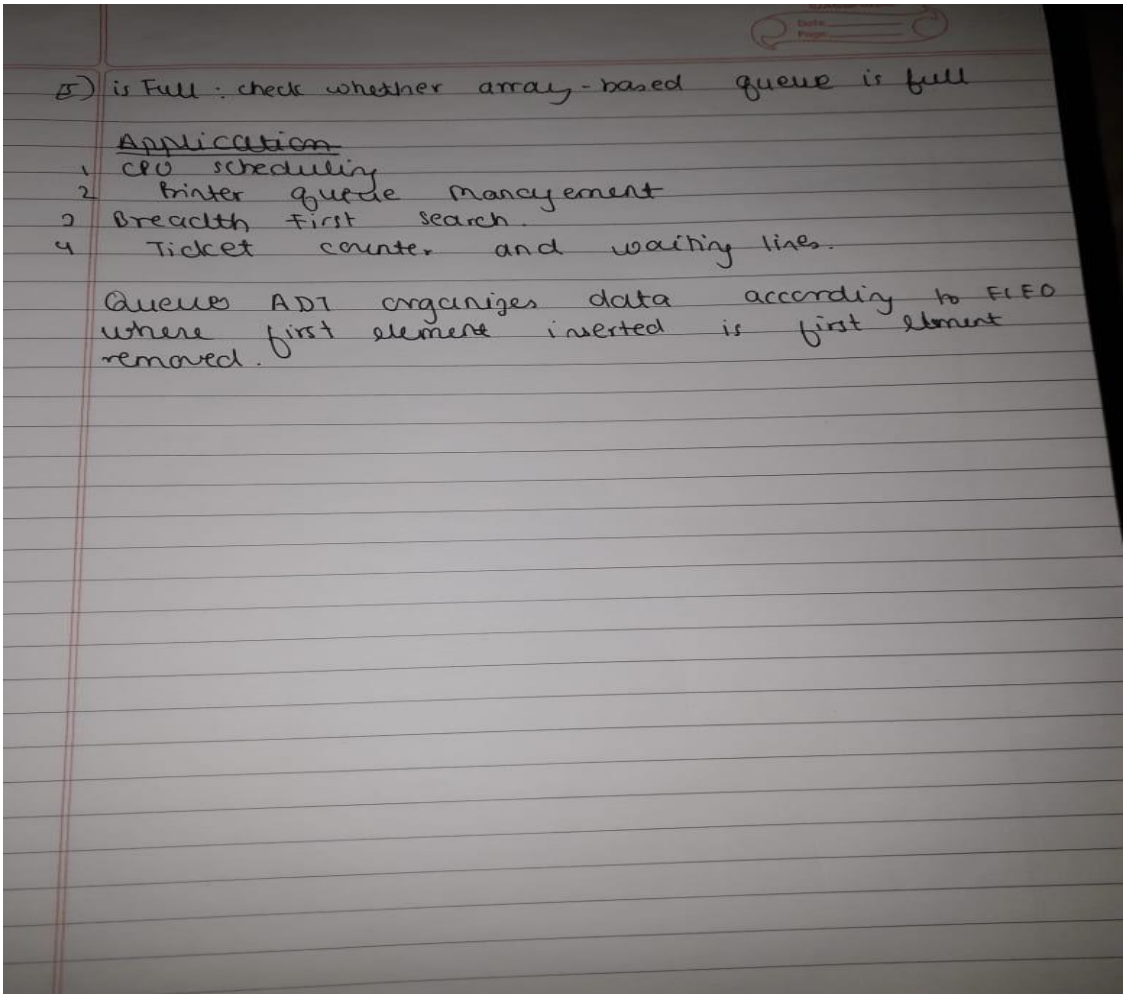
Basic opperation.
1 Push : Add element to top of stack
2 Pop : Remove top element from stack.
3 Peek/Top : Returns top element without removing it
4 Is Empty : Check whether stack is empty.
5 Is full : Check whether array-based stack is full

Application
1) Function calls & recursion
2) Undo & redo oper^n
3) Expression Evaluation.
4) Reversing string.

So it is useful for storing and manage data according to LIFO principle.

3) Queue ADT
→ It is linear data structure in which insertion is performed at the rear and deletion is performed at front It follows fifo.

Basic operation.
1) enqueue.
2) dequeue
3) Front/Peek : Return front element without removing
4) isEmpty : check whether queue is empty.



E) isFull : check whether array-based queue is full

Application
1. CPU scheduling
2. Printer queue management
3. Breadth first search.
4. Ticket counter and waiting lines.

Queues ADT organizes data according to FIFO where first element inserted is first element removed.